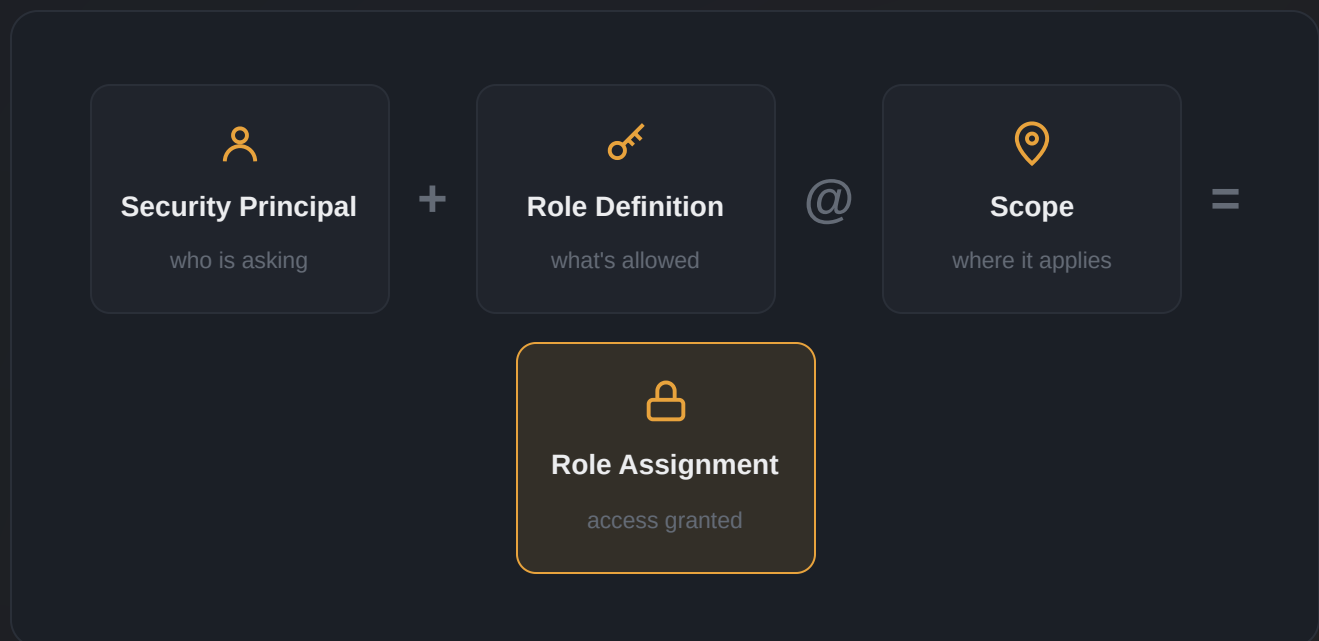


Azure RBAC

Role-Based Access Control, end to end: the core vocabulary, the decision flow for granting access, every tool you can use, and the mistakes that show up on the exam.

Why RBAC exists

RBAC is Azure's authorization layer: it decides *who* can do *what* to *which* resources. The goal is the principle of least privilege — every identity gets exactly the access it needs, nothing more — so a breached account, a mis-clicked delete, or an over-permissioned script does the smallest possible amount of damage. It also gives you an audit trail of who has access and why.



Eight terms you need cold

These show up in nearly every AZ-104 identity question, usually disguised as a scenario rather than asked directly.

Security Principal

An identity requesting access: a user, a group, a service principal (an app's identity), or a managed identity (an Azure resource's own identity).

Tip: Assign roles to groups, not individual users — it's the only way this scales.

Role Definition

A JSON collection of permissions: `Actions` (management ops allowed), `NotActions` (excluded), `DataActions` / `NotDataActions` (access to data inside the resource).

Tip: `Actions` = control plane. `DataActions` = the data itself (blobs, queue messages, etc).

Scope

The boundary a role applies to. Four levels, largest to smallest: Management Group → Subscription → Resource Group → Resource.

Tip: Access at a parent scope is inherited by every child beneath it.

Role Assignment

The actual grant: a Security Principal + Role Definition, applied at a Scope. This is the only object that gives access in Azure RBAC.

Deny Assignment

A rule that blocks specific actions for a principal, no matter what role assignments exist. Deny always overrides Allow.

Tip: Mostly auto-generated by Blueprints / Managed Apps, not hand-built by admins.

Custom Role

A role definition you build yourself from specific `Actions` / `DataActions` when no built-in role fits exactly.

Tip: Reach for built-in roles first — custom roles are more to maintain and audit.

Inheritance

Role assignments flow downward through scope. A role assigned at the subscription applies to every resource group and resource inside it.

Effective Permissions

The union of every role assignment that applies to a principal across all containing scopes — RBAC is additive, except where a deny assignment overrides it.

Built-in roles cheat sheet

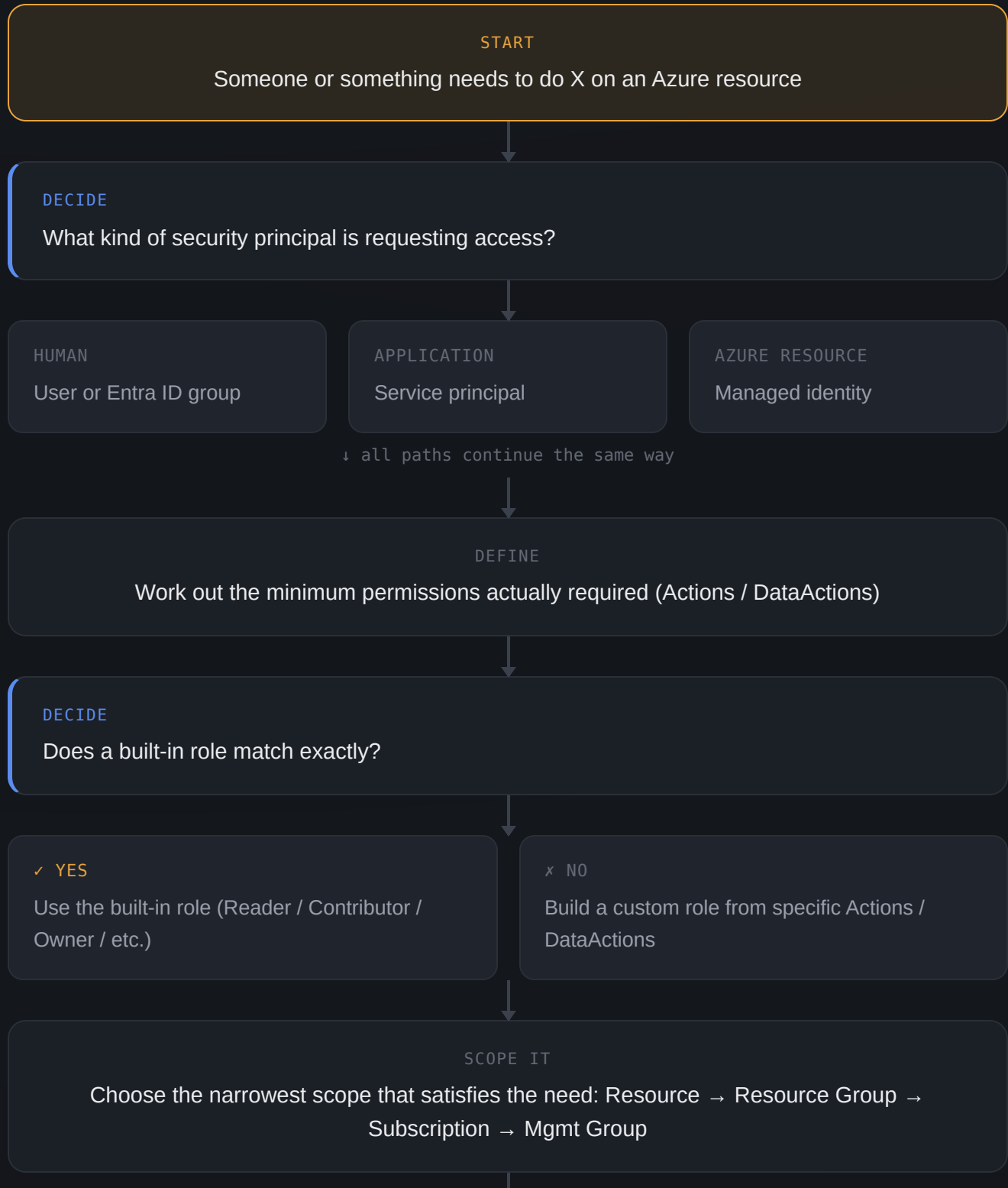
ROLE	GRANTS	CAN MANAGE ACCESS?
Owner	Full access to all resources, including delegating access to others	Yes
Contributor	Full access to manage resources — cannot grant access to others	No
Reader	View existing resources only	No
User Access Administrator	Manage user access to resources — cannot manage the resources themselves	Yes (only this)

EXAM GOTCHA

Azure RBAC ≠ Entra ID roles. Azure RBAC controls access to Azure resources (VMs, storage, networks) through Resource Manager. Entra ID roles control access to identity objects (users, groups, app registrations, licenses) — a completely separate system. Being a **Global Administrator** does not automatically make you an **Owner** on any subscription, though a Global Admin can elevate themselves to User Access Administrator at the root scope when needed.

Granting access, step by step

Walk through this every time you're deciding how to give someone (or something) access to an Azure resource.



DECIDE

Is this a one-off change, a script, or part of an IaC deployment?

ONE-OFF

Azure Portal → IAM blade

SCRIPTED

Azure CLI or PowerShell

IAC

Bicep / ARM template

CREATE

Role Assignment = Security Principal + Role Definition @ Scope

DECIDE

Should this access be permanent, or only active when needed?

ALWAYS-ON

Standard role assignment

JUST-IN-TIME

PIM eligible assignment (requires Entra ID P2)

VERIFY

Confirm with Portal "Check access" or `az role assignment list`

DONE

Access granted, scoped, and auditable

Five ways to assign a role

Same underlying action — a Role Assignment object — five different ways to create it. Pick based on whether it's a one-off, a script, or part of a repeatable deployment.

Azure Portal

BEST FOR: one-off changes, visual checks

No setup required, and the only tool with a built-in "Check access" simulator that shows exactly what a given user can actually do — the fastest way to learn and to troubleshoot.

```
Resource → Access control (IAM)  
→ Add → Add role assignment
```

Azure CLI

BEST FOR: scripting, CI/CD pipelines

Cross-platform, easy to drop into bash scripts or pipeline YAML, and simple to template across many subscriptions.

```
az role assignment create \  
  --assignee "user@domain.com" \  
  --role "Contributor" \  
  --scope "/subscriptions/<sub-id>/resourceGroups/<rg-name>"
```

> Azure PowerShell

BEST FOR: Windows-centric / enterprise scripting

Fits naturally alongside the rest of the Az PowerShell module if that's already how your environment is automated.

```
New-AzRoleAssignment `
  -SignInName "user@domain.com" `
  -RoleDefinitionName
"Contributor" `
  -ResourceGroupName "rg-name"
```

<> Bicep / ARM

BEST FOR: infrastructure as code, GitOps

Role assignments deploy in the same template as the resources they protect — nothing to forget, and every change goes through a pull request.

```
resource roleAssignment
'Microsoft.Authorization/roleAssi
gnments@2022-04-01' = {
  name: guid(resourceGroup().id,
principalId, roleId)
  properties: {
    roleDefinitionId:
subscriptionResourceId(

'Microsoft.Authorization/roleDefi
nitions', roleId)
    principalId: principalId
  }
}
```

🌐 REST API

BEST FOR: custom tooling

Every other tool on this list calls this API underneath. Reach for it when building your own integrations outside Microsoft's standard tooling.

```
PUT
https://management.azure.com/{sco
pe}/
providers/Microsoft.Authorization
/
roleAssignments/{roleAssignmentId
}
?api-version=2022-04-01
```

TOOL	BEST FOR	REPEATABLE	LEARNING CURVE
Portal	Learning & one-off changes	No	Low
CLI	Scripting / CI-CD	Yes	Medium
PowerShell	Windows / enterprise scripting	Yes	Medium
Bicep / ARM	IaC, GitOps	Yes	Medium–High
REST API	Custom integrations	Yes	High

Principles worth internalizing

Least privilege, narrowest scope, assign to groups. Almost every RBAC best practice is a variation on those three ideas.

RBAC VS. POLICY VS. LOCKS

This trio gets confused constantly on the exam — they solve three different problems and stack independently.

MECHANISM	CONTROLS	EXAMPLE
RBAC	Who can do what	Contributor on a resource group
Azure Policy	What configurations are allowed, regardless of who you are	Deny creation of resources outside West US
Resource Lock	Prevents accidental delete / modify, overriding even Owner	CanNotDelete lock on a production VNet

JUST-IN-TIME ACCESS – PIM

For roles that should be elevated only when actually needed, **Privileged Identity Management** lets you make an assignment "eligible" instead of "active." The user activates it for a limited time window, optionally with approval and MFA required. **PIM requires Microsoft Entra ID P2 licensing.**

Five mistakes that show up on the exam

Granting Owner when Contributor would do

Owner also grants the ability to change who else has access — almost never actually needed for day-to-day work.

Assigning at too broad a scope

A single app only needs its resource group, not the whole subscription. Broad scope means broad blast radius.

Expecting Contributor to manage access

It can't — only Owner and User Access Administrator can create role assignments for others.

Confusing RBAC with Entra ID roles

Global Administrator (Entra ID) does not equal Owner (Azure RBAC) — they're separate permission systems.

Forgetting deny assignments

A deny assignment silently blocks an action no matter how many role assignments say "yes." Deny always wins.

Assuming instant propagation

Role assignment changes can take up to ~30 minutes to fully propagate due to caching — not necessarily a misconfiguration.